

Home Assignment Report

Vibe Coding: Adding Playwright TypeScript Support to an AI Browser Extension

Objective: Use AI agent mode to understand an existing Chrome extension codebase and extend it with Playwright TypeScript code generation, using the existing Selenium Java implementation as the reference pattern.

Final result: The extension successfully generated a Playwright TypeScript Page Object for the SauceDemo login page.

1. Assignment Requirement

The assignment was to implement Playwright TypeScript or another automation tool using vibe coding. I chose Playwright TypeScript and integrated it into my existing AI Browser Extension for code generation.

Ask mode prompt: Read my entire workspace and confirm which files will be impacted if I want to add Playwright TypeScript the same way Selenium Java is implemented.

Agent mode prompt: Read the entire workspace, understand how Selenium Java is implemented, and add Playwright TypeScript support by updating the prompt templates, prompt selection logic, and UI logic.

2. Existing Project Overview

The project is a Manifest V3 Chrome Extension. It helps automation engineers generate code directly from a browser side panel by using selected DOM elements from the current web page.

Area	Purpose
manifest.json	Extension metadata, permissions, side panel, background worker, and web-accessible resources.
bg.js	Background service worker for extension lifecycle and side-panel behavior.
panel.html	User interface for selecting generation mode such as feature file and page class.
src/scripts/prompts.js	Stores prompt templates used for generating automation code.
src/scripts/chat.js	Controls prompt selection, chat flow, rendering, API communication, and validation.
src/styles	CSS for the extension side panel and chat UI.
assets/images	Extension icons and visual assets.

Screenshot: codebase explanation and workspace structure

ai-extension

manifest.json

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

```
{
  "manifest_version": 3,
  "name": "AI Browser Extension for Code Generation",
  "version": "1.0",
  "description": "Test automation code generator",
  "permissions": [
    "storage",
    "activeTab",
    "scripting",
    "tabs",
    "sidePanel"
  ],
  "host_permissions": [
    "<all_urls>"
  ],
  "action": {
    "default_title": "Extension for AI Code Generation",
    "default_icon": {
      "16": "assets/images/tets.png",
      "32": "assets/images/tets.png",
      "48": "assets/images/tets.png",
      "128": "assets/images/tets.png"
    }
  },
  "icons": {
    "16": "assets/images/tets.png",
    "32": "assets/images/tets.png",
    "48": "assets/images/tets.png",
    "128": "assets/images/tets.png"
  },
  "side_panel": {
    "default_path": "panel.html"
  },
  "background": {
    "service_worker": "bg.js"
  },
  "web_accessible_resources": [
    {
      "resources": [
        "lib/marked/marked.min.js",
```

Explaining AI Extension Codebase

Can you tell me and explain my codebase

Worked for 48s >

This codebase is a **Manifest V3 Chrome Extension** designed to assist automation test engineers in generating test automation code (such as Selenium Page Object Classes, Cucumber Feature files, and Step Definitions) directly from a browser side-panel, using selected live DOM elements from any web page.

High-Level Directory Structure

ai-extension/

├─ manifest.json

├─ script registration

├─ bg.js

├─ panel.html

├─ assets/

├─ lib/

├─ (Prism, Marked, FontAwesome)

├─ src/

├─ config/

├─ (appConfig.js, configUtils.js)

├─ content/

├─ (content.js)

├─ styles/

├─ (sidepanel.css, etc.)

├─ scripts/

├─ api/

├─ Testleaf

├─ chat.js

Extension metadata, permissions, and script registration

Service worker running in the background

Side-panel UI structure (Chat & Settings tabs)

Extension icons and visual assets

Third-party styling & formatting libraries (Prism, Marked, FontAwesome)

Dynamic configurations & style variables (appConfig.js, configUtils.js)

Content scripts injected into the webpage (content.js)

Sidebar & panel stylesheets (chat.css, sidepanel.css, etc.)

Application controller scripts

LLM API connection clients (Groq, OpenAI, Testleaf)

Controls chat history, prompts, rendering, and API communication

Ask anything, @ to mention, / for actions

+ GPT-OSS 120B (Medium) ^

3. Files Impacted by the Implementation

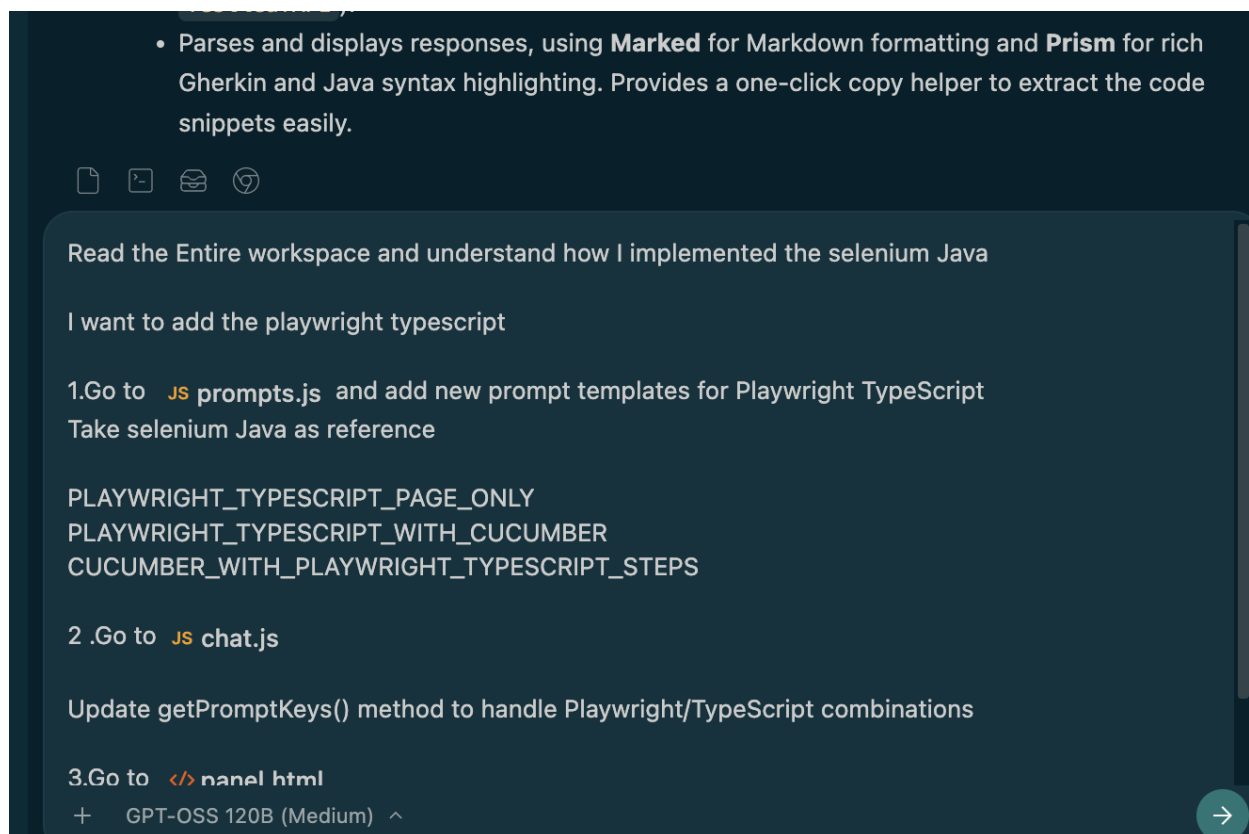
After analysing the workspace, the AI identified the main files that needed modification. The implementation affected these files:

File	Change Made
src/scripts/prompts.js	Added three new prompt templates for Playwright TypeScript generation.
src/scripts/chat.js	Updated prompt-key selection logic so the extension can choose Playwright TypeScript prompts based on language and engine.
panel.html	Added UI logic/options so Playwright TypeScript can be selected and handled similar to Selenium Java.

4. Agent Mode Instructions Used

I used agent mode with clear implementation steps. The agent was instructed to use Selenium Java as the reference implementation and add equivalent support for Playwright TypeScript.

Screenshot: agent mode prompt and changes applied



The agent edited prompts.js and chat.js, adding the new Playwright TypeScript templates and helper logic for supported language/engine combinations.

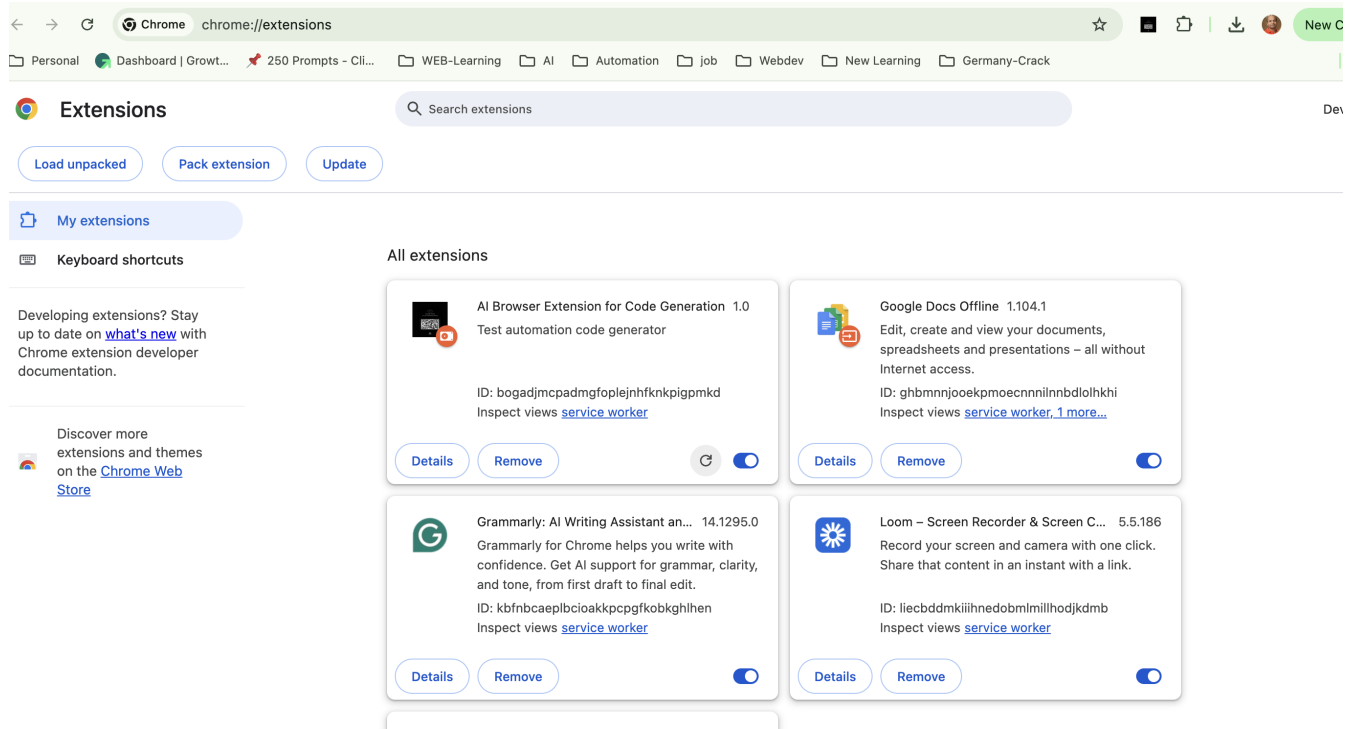
5. Prompt Templates Added

The following prompt keys were added in `src/scripts/prompts.js`:

- `PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY`
- `PLAYWRIGHT_TYPESCRIPT_WITH_CUCUMBER`
- `CUCUMBER_WITH_PLAYWRIGHT_TYPESCRIPT_STEPS`

These templates allow the extension to generate: a Page Object class only, a Playwright TypeScript solution with Cucumber, and Cucumber step definitions using Playwright TypeScript.

Screenshot: `prompts.js` updated with Playwright TypeScript templates



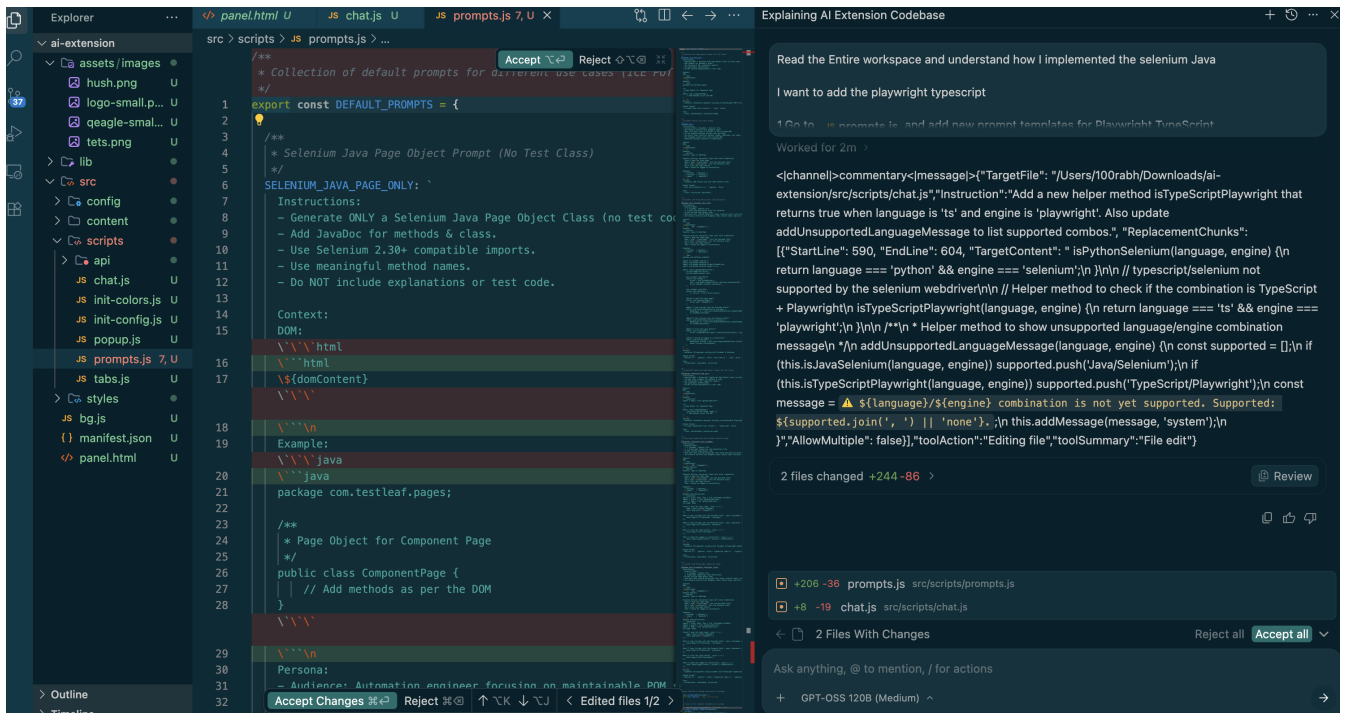
6. Logic Updated in `chat.js`

The `getPromptKeys()` flow was updated so the extension can choose the right prompt when the user selects TypeScript and Playwright. A helper method was also added to validate the TypeScript + Playwright combination.

This made the new Playwright path behave consistently with the existing Selenium Java path.

7. Extension Loaded in Chrome

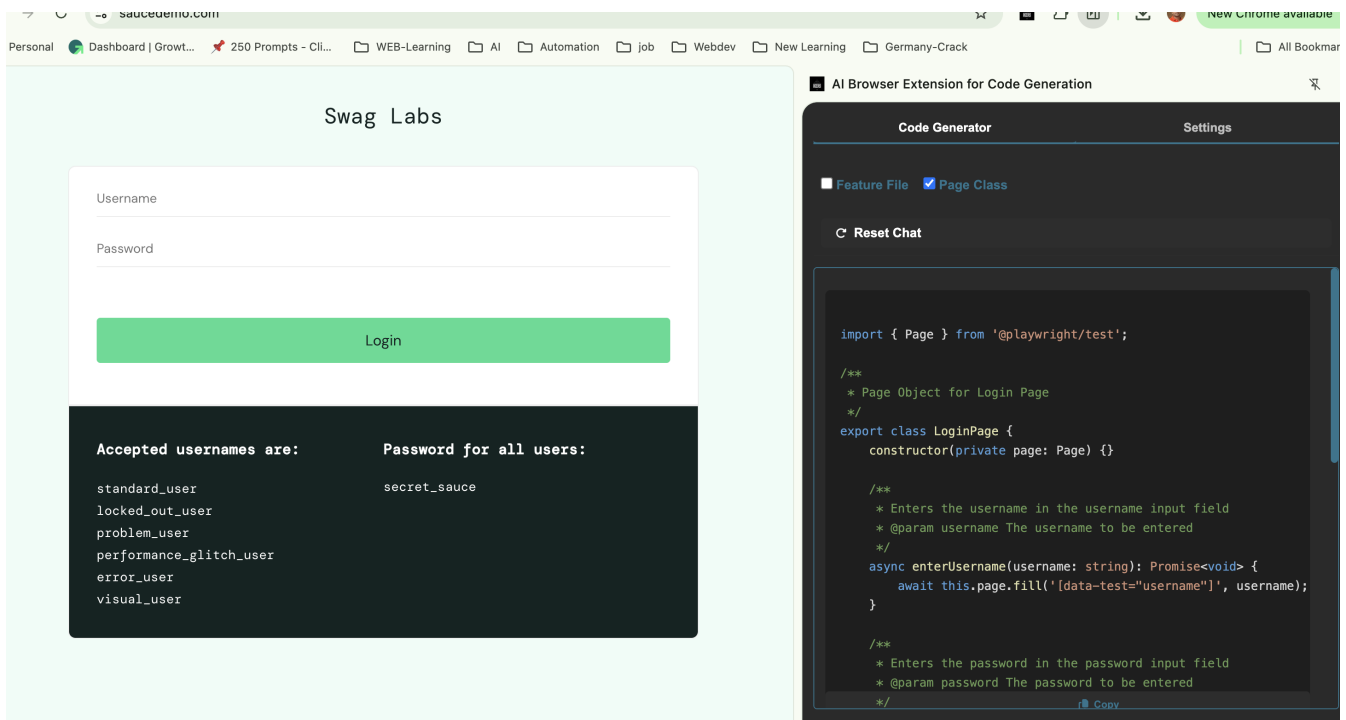
After the changes were applied, I loaded the extension in Chrome using developer mode. The extension appeared as AI Browser Extension for Code Generation.



8. Testing on SauceDemo

To validate the implementation, I opened saucedemo.com and used the extension side panel to generate a Playwright TypeScript Page Object for the login page.

Screenshot: Playwright TypeScript output generated in the extension



Generated Playwright TypeScript Page Object

```
import { Page } from '@playwright/test';

/**
 * Page Object for Login Page
 */
export class LoginPage {
  constructor(private page: Page) {}

  /**
   * Enters the username in the username input field
   * @param username The username to be entered
   */
  async enterUsername(username: string): Promise<void> {
    await this.page.fill('[data-test="username"]', username);
  }

  /**
   * Enters the password in the password input field
   * @param password The password to be entered
   */
  async enterPassword(password: string): Promise<void> {
    await this.page.fill('[data-test="password"]', password);
  }

  /**
   * Clicks the login button
   */
  async clickLoginButton(): Promise<void> {
    await this.page.click('[data-test="login-button"]');
  }

  /**
   * Gets the username from the username input field
   * @returns The username
   */
  async getUsername(): Promise<string> {
    return await this.page.inputValue('[data-test="username"]');
  }

  /**
   * Gets the password from the password input field
   * @returns The password
   */
}
```

```
    */  
    async getPassword(): Promise<string> {  
        return await this.page.inputValue('[data-test="password"]');  
    }  
}
```

9. What I Learned

- How to use AI ask mode to analyse a codebase before making changes.
- How to use agent mode to make targeted code changes across multiple files.
- How prompt templates control code generation output in an AI extension.
- How to reuse an existing Selenium Java implementation pattern to add a new Playwright TypeScript path.
- How to test a browser extension by loading it locally and verifying generated automation code on a real web page.

10. Conclusion

I successfully completed the assignment by extending my AI Browser Extension from Selenium Java support to Playwright TypeScript support. The implementation added new prompt templates, updated prompt selection logic, and validated the output by generating a working Playwright TypeScript Page Object for the SauceDemo login page.